

Growth Function

(Unit-10)

Algorithm refers to a set of instructions or rules that are followed to solve a particular problem. Algorithms are used in computer programming to solve problems by breaking them down into smaller, more manageable steps that can be executed by a computer. It may involve various operations such as mathematical computations, logical decisions, comparisons, iterations, and data manipulation to achieve the desired output.

Characteristics of algorithm

1. Unambiguous

This means that there is no room for interpretation or ambiguity in the instruction, the algorithm should be clear and precise, so that it can be implemented correctly and produce the desired output.

2. Input

The input is the data that is provided to the algorithm.

3. Output

The output is the result of the algorithm.

4. Effectiveness

Effectiveness refers to the ability of the algorithm to solve the problem in a reasonable amount of time and with a reasonable amount of resources. This includes considerations such as the running time of the algorithm and the amount of memory it uses.

1. Time Complexity

Time complexity is a measure of how long an algorithm takes to run. It is usually expressed as a function of the size of the input data.

2. Space Complexity

Time complexity is a measure of how long an algorithm takes to run. It is usually expressed as a function of the size of the input data.

5. Finiteness - For all different cases, the algorithm must produce result within a finite number of steps.

When designing an algorithm, it is important to consider the following factors:

- **The complexity of the problem:** The algorithm should be designed to be as simple as possible, while still being able to solve the problem.
- **The size of the input data:** The algorithm should be designed to be efficient, so that it can run quickly even with large input data.

- **The accuracy of the algorithm:** The algorithm should be designed to be accurate, so that it produces the correct results.

What is Performance Analysis of an algorithm?

When we have more than one algorithm to solve a problem (in computer science, there are multiple algorithms to solve a problem.), we need to select the best one. Performance analysis helps us to select the best algorithm from multiple algorithms to solve a problem.

When there are multiple alternative algorithms to solve a problem, we analyze them and pick the one which is best suitable for our requirements.

Performance of an algorithm is a process of making evaluative judgement about algorithms. In other word, Performance of an algorithm means predicting the resources which are required to an algorithm to perform its task.

To compare algorithms, we use a set of parameters or set of elements like **memory required by that algorithm**, the **execution speed of that algorithm**, easy to understand, easy to implement, etc.,

When we want to analyze an algorithm, we consider only the space and time required by that particular algorithm and we ignore all the remaining elements.

Performance analysis of an algorithm is performed by using the following measures...

- Space required to complete the task of that algorithm (**Space Complexity**). It includes program space and data space
- Time required to complete the task of that algorithm (**Time Complexity**)

Time complexity

Every algorithm requires some amount of computer time to execute its instruction to perform the task. This computer time required is called time complexity. Time complexity is a measure of how long an algorithm takes to run.

Generally, the running time of an algorithm depends upon the following...

- Whether it is running on Single processor machine or Multi processor machine.
- Whether it is a 32 bit machine or 64 bit machine.
- Read and Write speed of the machine.
- The amount of time required by an algorithm to perform Arithmetic operations, logical operations, return value and assignment operations etc.,
- Input data

When we calculate time complexity of an algorithm, **we consider only input data and ignore the remaining things**, as they are machine dependent. It is usually expressed as a function of the

size of the input data. For example, an algorithm with time complexity $O(n)$ will take time proportional to the size of the input data.

Let us consider 2 developers created a program to sort n numbers and the time taken is as follow:

n	Running time on A	Running time on B
10	2 sec	~ 1 h
100	20 sec	~ 1.8 h
10^6	~ 55.5 h	~ 5.5 h
10^9	~ 6.3 years	~ 8.3 h

Which one is good?

Here, the algorithm by **A** works well for smaller number of elements. However, as we increase the number of elements, algorithm by **B** performs better.

Space Complexity

When we design an algorithm to solve a problem, it needs some computer memory to complete its execution. Space complexity is the amount of memory required by an algorithm to solve a problem. It is also influenced by the input size.

Total amount of computer memory required by an algorithm to complete its execution is called as space complexity of that algorithm.

For any algorithm, memory is required for the following purposes...

- To store program instructions.
- To store constant values.
- To store variable values.
- And for few other things like function calls, jumping statements etc.,.

To calculate the space complexity, we must know the memory required to store different datatype values (according to the compiler). For example, the C Programming Language compiler requires the following...

- 2 bytes to store Integer value.

- 1 byte to store Character value.

In general, the space complexity of an algorithm is related to the **amount of data that it needs to store in memory during its execution**. This data can be either input data, intermediate results, or auxiliary structures such as arrays or linked lists.

When analyzing an algorithm's time and space complexity, we often consider three different cases:

- **the best case,**
- **worst case,**
- **and average case.**

Best case: The best case represents the situation in which the algorithm performs the best or requires the minimum amount of time or space to complete its task. This scenario generally occurs when the input is already sorted, or the search element is found at the beginning of the list. The best-case time complexity of an algorithm is typically represented by the lower bound of the Big O notation.

Worst case: The worst case represents the situation in which the algorithm performs the worst or requires the maximum amount of time or space to complete its task. This scenario generally occurs when the input is in reverse order, or the search element is not present in the list. The worst-case time complexity of an algorithm is typically represented by the upper bound of the Big O notation.

Average case: The average case represents the scenario that is most likely to happen when running the algorithm on random input data. The average-case time complexity of an algorithm is typically represented by the expected time taken to complete the task, where the expectation is taken over all possible input data.

Running Time Calculation

Asymptotic analysis

Asymptotic analysis is a way of analyzing the efficiency of an algorithm as a function of the size of its input. It is used to compare the efficiency of different algorithms for solving the same problem. Asymptotic analysis provides a way to evaluate the performance of an algorithm by studying its behavior as the input size grows towards infinity.

Asymptotic notation is a mathematical notation that is used to represent the asymptotic growth rate of a function.

Asymptotic notation of an algorithm is a mathematical representation of its complexity.

Mathematical notation for algorithm analysis, particularly asymptotic notation, provides a standardized way to express the performance of an algorithm in terms of its time and space complexity. By using mathematical notation, we can precisely quantify the growth rate of an

algorithm as the input size increases, making it easier to compare and evaluate different algorithms.

Here are some of the benefits of using asymptotic analysis:

- It can be used to compare the performance of different algorithms.
- It can be used to choose the best algorithm for a given problem.
- It can be used to improve the performance of algorithms.
- It can be used to understand the behavior of algorithms as their input sizes grow.

There are three main types of asymptotic notation:

- Big O notation (O)
- Omega notation (Ω)
- Theta notation (Θ)

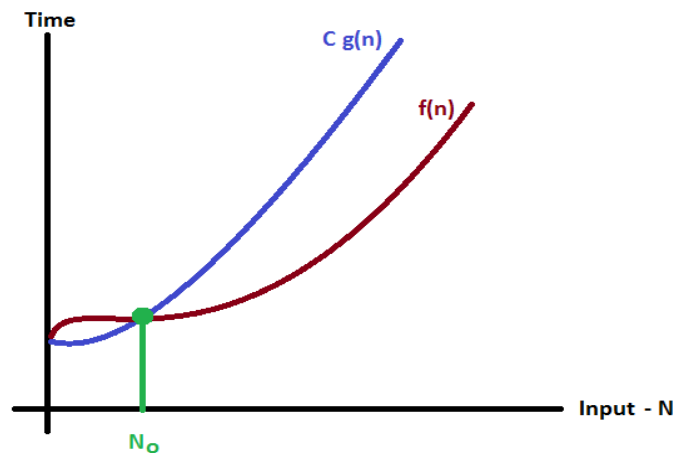
1. Big - Oh Notation (O)

Big - Oh notation is used to define the upper bound of an algorithm in terms of Time Complexity. That means Big - Oh notation always indicates the maximum time required by an algorithm for all input values. That means Big - Oh notation describes the worst case of an algorithm time complexity. If a function is $O(n)$, it is automatically $O(n^2)$ as well.

Big - Oh Notation can be defined as follows...

- Consider function $f(n)$ as time complexity of an algorithm and $g(n)$ is the most significant term. If $f(n) \leq C g(n)$ for all $n \geq n_0$, $C > 0$ and $n_0 \geq 1$. Then we can represent $f(n)$ as $O(g(n))$.
- $f(n) = O(g(n))$

Consider the following graph drawn for the values of $f(n)$ and $C g(n)$ for input (n) value on X-Axis and time required is on Y-Axis.



In above graph after a particular input value n_0 , always $C g(n)$ is greater than $f(n)$ which indicates the algorithm's upper bound.

Example:-

Consider the following $f(n)$ and $g(n)$...

$$f(n) = 3n + 2$$

$$g(n) = n$$

If we want to represent $f(n)$ as $O(g(n))$ then it must satisfy $f(n) \leq C g(n)$ for all values of $C > 0$ and $n_0 \geq 1$

$$f(n) \leq C g(n) \Rightarrow 3n + 2 \leq C n$$

Above condition is always TRUE for all values of $C = 4$ and $n \geq 2$.

By using Big - Oh notation we can represent the time complexity as follows...

$$3n + 2 = O(n)$$

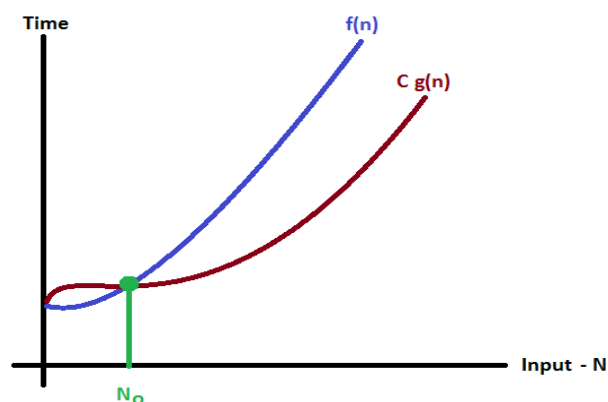
Big – Omega Notation (Ω)

Big - Omega notation is used to define the lower bound of an algorithm in terms of Time Complexity. That means Big-Omega notation always indicates the minimum time required by an algorithm for all input values. That means Big-Omega notation describes the best case of an algorithm time complexity. If a function is $\Omega(n^2)$, it is automatically $\Omega(n)$.

Big - Omega Notation can be defined as follows...

- Consider function $f(n)$ as time complexity of an algorithm and $g(n)$ is the most significant term. If $f(n) \geq C g(n)$ for all $n \geq n_0$, $C > 0$ and $n_0 \geq 1$. Then we can represent $f(n)$ as $\Omega(g(n))$.
- $f(n) = \Omega(g(n))$

Consider the following graph drawn for the values of $f(n)$ and $C g(n)$ for input (n) value on X-Axis and time required is on Y-Axis



In above graph after a particular input value n_0 , always $C g(n)$ is less than $f(n)$ which indicates the algorithm's lower bound.

Example:-

Consider the following $f(n)$ and $g(n)$...

$$f(n) = 3n + 2$$

$$g(n) = n$$

If we want to represent $f(n)$ as $\Omega(g(n))$ then it must satisfy $f(n) \geq C g(n)$ for all values of $C > 0$ and $n_0 \geq 1$

$$f(n) \geq C g(n) \Rightarrow 3n + 2 \geq C n$$

Above condition is always TRUE for all values of $C = 1$ and $n \geq 1$.

By using Big - Omega notation we can represent the time complexity as follows...

$$3n + 2 = \Omega(n)$$

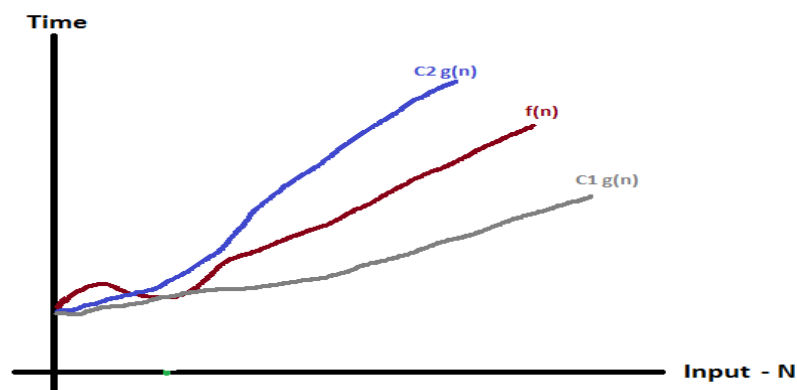
Big - Theta Notation (Θ)

Big - Theta notation is used to define the average bound of an algorithm in terms of Time Complexity. That means Big - Theta notation always indicates the average time required by an algorithm for all input values. That means Big - Theta notation describes the average case of an algorithm time complexity.

Big - Theta Notation can be defined as follows...

- Consider function $f(n)$ as time complexity of an algorithm and $g(n)$ is the most significant term. If $C_1 g(n) \leq f(n) \leq C_2 g(n)$ for all $n \geq n_0$, $C_1 > 0$, $C_2 > 0$ and $n_0 \geq 1$. Then we can represent $f(n)$ as $\Theta(g(n))$.
- $f(n) = \Theta(g(n))$

Consider the following graph drawn for the values of $f(n)$ and $C g(n)$ for input (n) value on X-Axis and time required is on Y-Axis



In above graph after a particular input value n_0 , always $C_1 g(n)$ is less than $f(n)$ and $C_2 g(n)$ is greater than $f(n)$ which indicates the algorithm's average bound.

Example:-

Consider the following $f(n)$ and $g(n)$...

$$f(n) = 3n + 2$$

$$g(n) = n$$

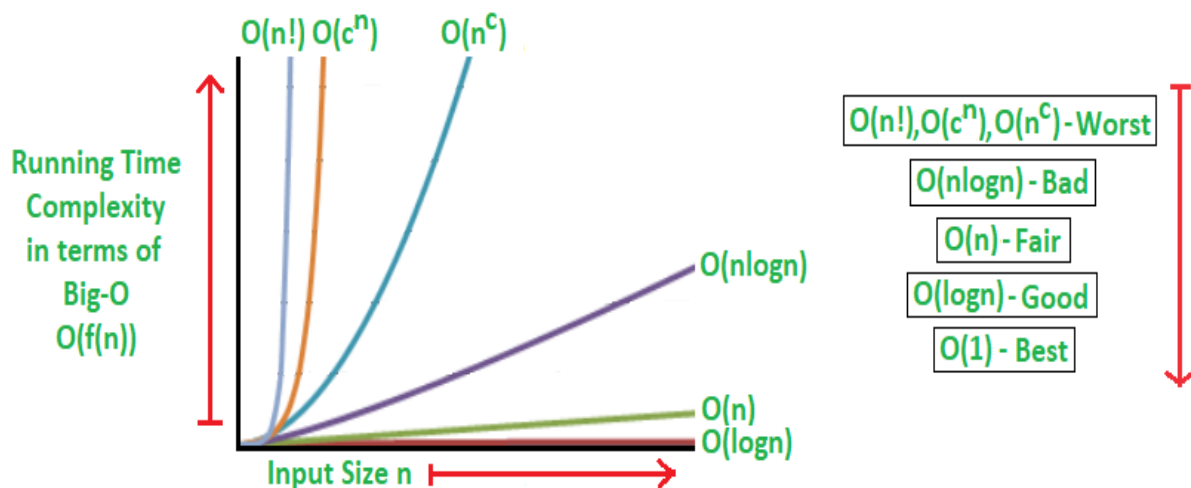
If we want to represent $f(n)$ as $\Theta(g(n))$ then it must satisfy $C_1 g(n) \leq f(n) \leq C_2 g(n)$ for all values of $C_1 > 0$, $C_2 > 0$ and $n_0 \geq 1$

$$C_1 g(n) \leq f(n) \leq C_2 g(n) \Rightarrow C_1 n \leq 3n + 2 \leq C_2 n$$

Above condition is always TRUE for all values of $C_1 = 1$, $C_2 = 4$ and $n \geq 2$.

By using Big - Theta notation we can represent the time complexity as follows...

$$3n + 2 = \Theta(n)$$



Common Asymptotic Notations

Following is a list of some common asymptotic notations –

constant	–	$O(1)$
logarithmic	–	$O(\log n)$
linear	–	$O(n)$
$n \log n$	–	$O(n \log n)$
quadratic	–	$O(n^2)$
cubic	–	$O(n^3)$
polynomial	–	$n^{O(1)}$
exponential	–	$2^{O(n)}$

Time Complexity or Time requirement

Let's suppose that each statement of the algorithm takes one unit of time. There are three statements in the given algorithm, but one statement consists of two operations add and assignment, so, total time take by the algorithm to execute is $1+(1+1)+1 = 4$ units of time. We can say that the time requirement of given algorithm is Order of 1, $O(1)$.

Space Complexity or Space Requirement

Here, in the given algorithm three variables: a, b, and s are declared. Each variable takes 1 unit of memory space, so total space (memory) requirement is $1 + 1 + 1 = 3$ units. It can be said that space requirement is of order 1, $O(1)$. If the time or space requirement for an algorithm is some constant like: 1, 5, 40 etc. then we say the time or space requirement is order of 1, $O(1)$.

function sum(int n,int[] Arr) {	
int s = 0; int i;	----- 1 + 1 = 2 units
for(i=0;i<n;i++){	----- $1 + (n+1)+n = (2n + 2)$ units; we can ignore smaller terms. If $n = 4$ then the condition of the for loop will run: when $i=0$; $i = 1$; $i = 2$; $i = 3$; $i = 4$; so, total number of times condition checking runs is $n + 1$.
s = s + Arr[i];	--n units (0, 1, 2, 3 if $n=4$) for displaying
}	

display s;	--- 1 unit of time
}	

The time complexity of the given algorithm which adds all the numbers of the given array of size n is $2 + (2n+2) + n + 1 = (3n + 5)$ Units. The degree of polynomial $2n+3$ is 1, so the algorithm is order of n , $O(n)$.

function sumArrays(int [] arra, int[] arrb){	
int s = 0,i,j;	----- 1 + 1 + 1 = 3 units
for(i=0;i<n;i++){	----- n+1 units (ignoring i=0; and i++)
for(j=0;j<n;j++){	-----n * n+1 units
s = arra[i] + arrb[j];	--n *n units for displaying
}	
}	
display s;	--- 1 unit of time
}	

The time complexity of the given algorithm sumArrays, is: $3 + (n+1) + (n * (n+1)) + (n * n) + 1 = 2n^2+2n+5$ polynomial is of degree 2, so, the algorithm is of order of n^2 , $O(n^2)$

Q) Why can't we calculate complexity in terms of seconds?

Limitations of Big O Notation

1.Ignoreing Constant Factors and Lower-Order Terms

One of the primary criticisms of Big O notation is that it disregards constant factors and lower-order terms. This means that two algorithms with the same Big O notation might have vastly different real-world performance due to these ignored factors. For small input sizes, these constants and lower-order terms can significantly affect the runtime.

2.Lack of Consideration for Hardware Specifics

Big O notation abstracts away the hardware specifics. It does not account for the architecture of the machine on which the algorithm runs, which can influence the actual performance. Factors such as CPU cache, memory speed, and parallel processing capabilities are not reflected in Big O analysis.

3.Theoretical Nature

Big O is theoretical and does not always represent practical performance. It's based on the assumption that we're only interested in the growth rate as the input size approaches infinity, which may not be practical for finite and often small input sizes encountered in real-world scenarios.